JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface ListIterator<E>

All Superinterfaces:

Iterator<E>

```
public interface ListIterator<E>
extends Iterator<E>
```

An iterator for lists that allows the programmer to traverse the list in either direction, modify the list during iteration, and obtain the iterator's current position in the list. A `ListIterator` has no current element; its *cursor position* always lies between the element that would be returned by a call to `previous()` and the element that would be returned by a call to `next()`. An iterator for a list of length `n` has `n+1` possible cursor positions, as illustrated by the carets (^) below:

```
                      Element(0)   Element(1)   Element(2)   ... Element(n-1)
cursor positions:  ^           ^           ^           ^                   ^
```

Note that the `remove()` and `set(Object)` methods are *not* defined in terms of the cursor position; they are defined to operate on the last element returned by a call to `next()` or `previous()`.

This interface is a member of the Java Collections Framework.

Since:

1.2

See Also:

Collection, List, Iterator, Enumeration, List.listIterator()

Method Summary

Abstract Methods

Modifier and Type	Method	Description
void	**add**(**E** e)	Inserts the specified element into the list (optional operation).
boolean	**hasNext**()	Returns `true` if this list iterator has more elements when traversing the list in the forward direction.
boolean	**hasPrevious**()	Returns `true` if this list iterator has more elements when traversing the list in the reverse direction.
E	**next**()	Returns the next element in the list and advances the cursor position.
int	**nextIndex**()	Returns the index of the element that would be returned by a subsequent call to **next()**.
E	**previous**()	Returns the previous element in the list and moves the cursor position backwards.
int	**previousIndex**()	Returns the index of the element that would be returned by a subsequent call to **previous()**.
void	**remove**()	Removes from the list the last element that was returned by **next()** or **previous()** (optional operation).
void	**set**(**E** e)	Replaces the last element returned by **next()** or **previous()** with the specified element (optional operation).

Methods inherited from interface java.util.Iterator

forEachRemaining

Method Detail

hasNext

```
boolean hasNext()
```

Returns `true` if this list iterator has more elements when traversing the list in the forward direction. (In other words, returns `true` if `next()` would return an element rather than throwing an exception.)

Specified by:

`hasNext` in interface `Iterator<E>`

Returns:

`true if the list iterator has more elements when traversing the list in the forward direction`

next

```
E next()
```

Returns the next element in the list and advances the cursor position. This method may be called repeatedly to iterate through the list, or intermixed with calls to `previous()` to go back and forth. (Note that alternating calls to `next` and `previous` will return the same element repeatedly.)

Specified by:

`next` in interface `Iterator<E>`

Returns:

`the next element in the list`

Throws:

`NoSuchElementException - if the iteration has no next element`

hasPrevious

```
boolean hasPrevious()
```

Returns `true` if this list iterator has more elements when traversing the list in the reverse direction. (In other words, returns `true` if `previous()` would return an element rather than throwing an exception.)

Returns:

`true if the list iterator has more elements when traversing the list in the reverse direction`

previous

`E previous()`

Returns the previous element in the list and moves the cursor position backwards. This method may be called repeatedly to iterate through the list backwards, or intermixed with calls to `next()` to go back and forth. (Note that alternating calls to `next` and `previous` will return the same element repeatedly.)

Returns:

`the previous element in the list`

Throws:

`NoSuchElementException - if the iteration has no previous element`

nextIndex

`int nextIndex()`

Returns the index of the element that would be returned by a subsequent call to `next()`. (Returns list size if the list iterator is at the end of the list.)

Returns:

`the index of the element that would be returned by a subsequent call to next, or list size if the list iterator is at the end of the list`

previousIndex

`int previousIndex()`

Returns the index of the element that would be returned by a subsequent call to `previous()`. (Returns -1 if the list iterator is at the beginning of the list.)

Returns:

`the index of the element that would be returned by a subsequent call to previous, or -1 if the list iterator is at the beginning of the list`

remove

`void remove()`

Removes from the list the last element that was returned by `next()` or `previous()` (optional operation). This call can only be made once per call to

next or previous. It can be made only if add(E) has not been called after the last call to next or previous.

Specified by:

remove in interface Iterator<E>

Throws:

UnsupportedOperationException - if the remove operation is not supported by this list iterator

IllegalStateException - if neither next nor previous have been called, or remove or add have been called after the last call to next or previous

set

void set(E e)

Replaces the last element returned by next() or previous() with the specified element (optional operation). This call can be made only if neither remove() nor add(E) have been called after the last call to next or previous.

Parameters:

e - the element with which to replace the last element returned by next or previous

Throws:

UnsupportedOperationException - if the set operation is not supported by this list iterator

ClassCastException - if the class of the specified element prevents it from being added to this list

IllegalArgumentException - if some aspect of the specified element prevents it from being added to this list

IllegalStateException - if neither next nor previous have been called, or remove or add have been called after the last call to next or previous

add

void add(E e)

Inserts the specified element into the list (optional operation). The element is inserted immediately before the element that would be returned by next(), if any, and after the element that would be returned by previous(), if any. (If the list contains no elements, the new element becomes the sole element on the list.) The new element is inserted before the implicit cursor: a subsequent call to next would be unaffected, and a subsequent call to

previous would return the new element. (This call increases by one the value that would be returned by a call to nextIndex or previousIndex.)

Parameters:

e - the element to insert

Throws:

UnsupportedOperationException - if the add method is not supported by this list iterator

ClassCastException - if the class of the specified element prevents it from being added to this list

IllegalArgumentException - if some aspect of this element prevents it from being added to this list

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.